

AI508

Computer

Vision

Peter Schneider-Kamp
<petersk@imada.sdu.dk>



Task of the day

Image classification

Object detection

Segmentation

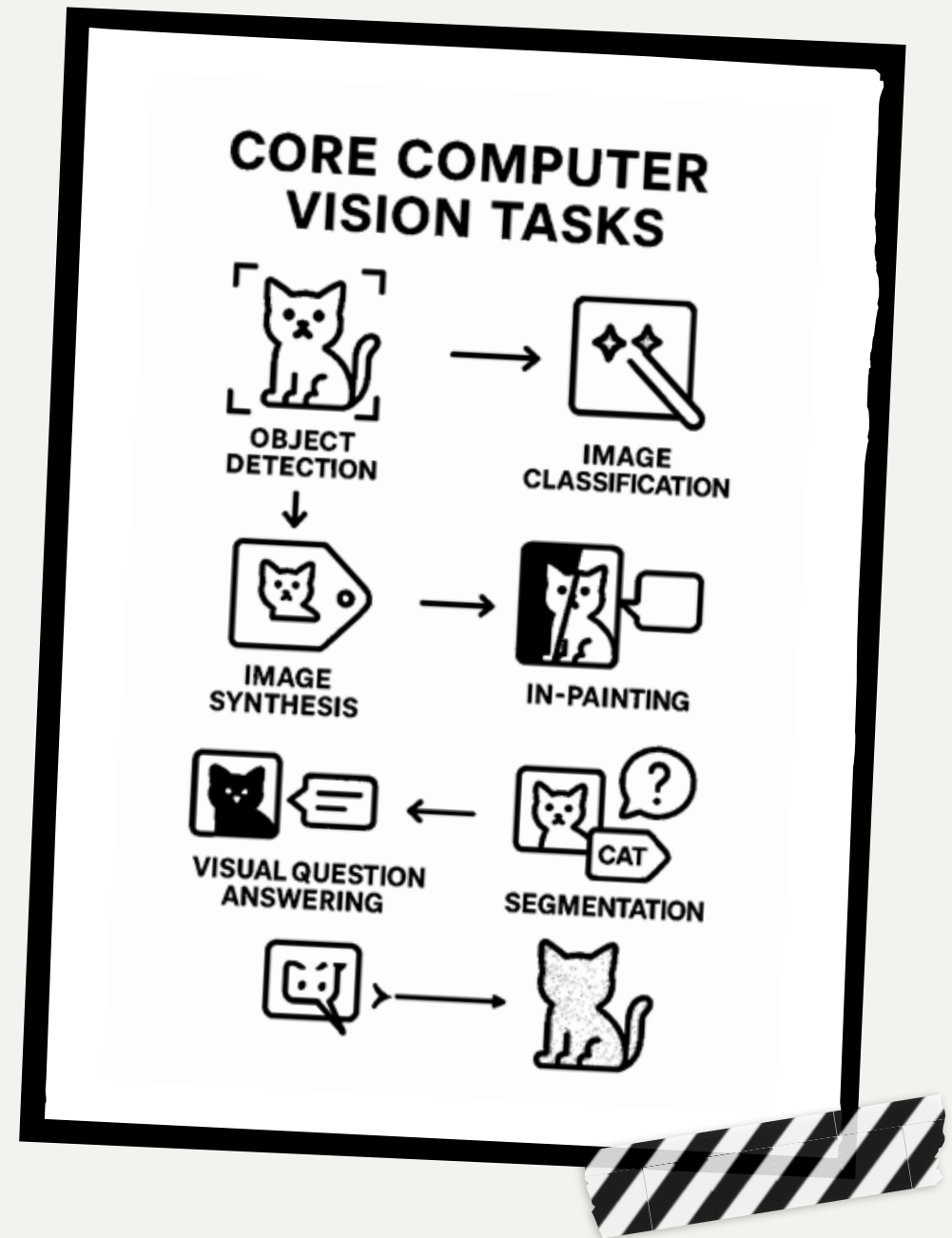
Image generation

Inpainting

Image-text retrieval

Visual question answering

Video generation



Object detection

GOAL :

Localize objects on an image.

METRICS:

IoU, mean Average Precision.

DATASETS:

COCO, VOC.

ARCHITECTURES:

R-CNN, YOLO.



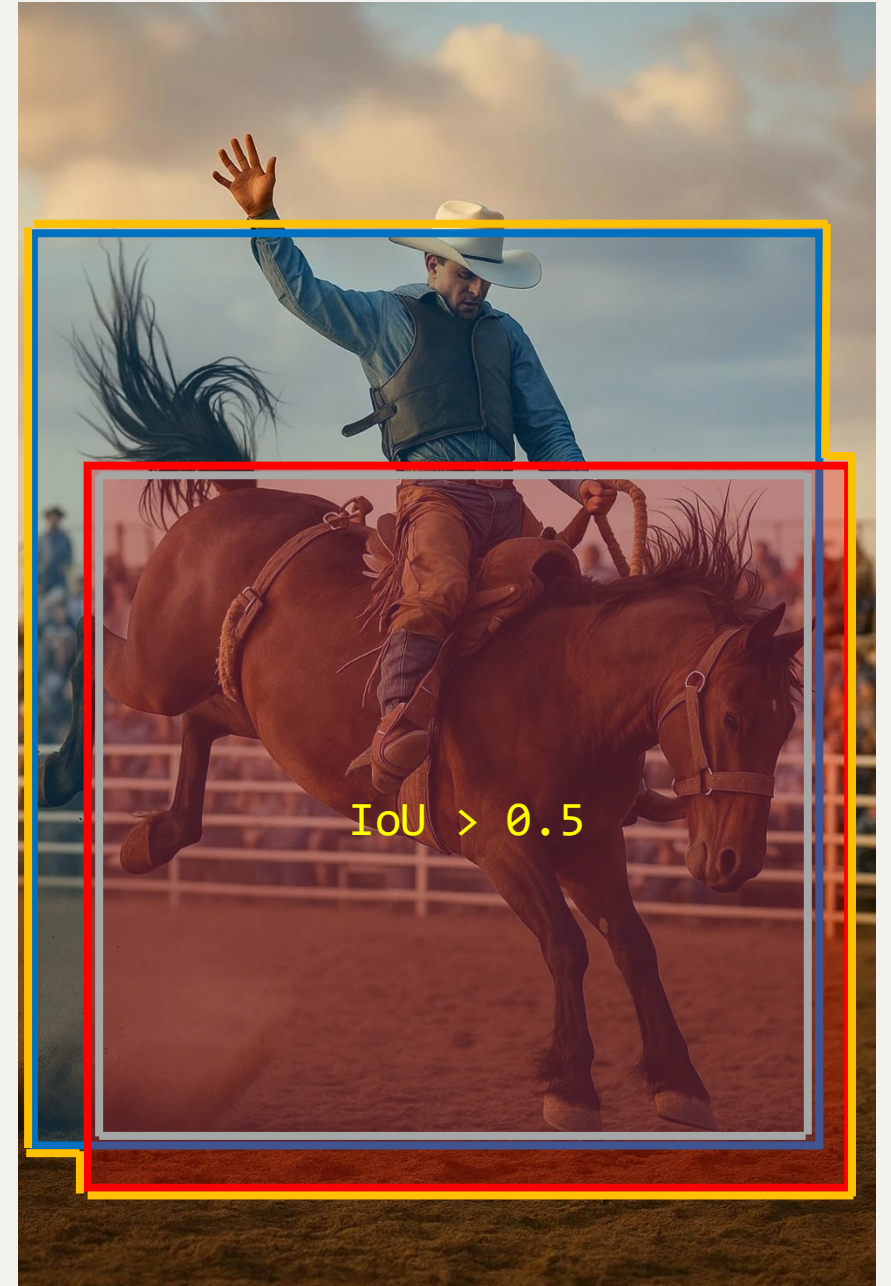
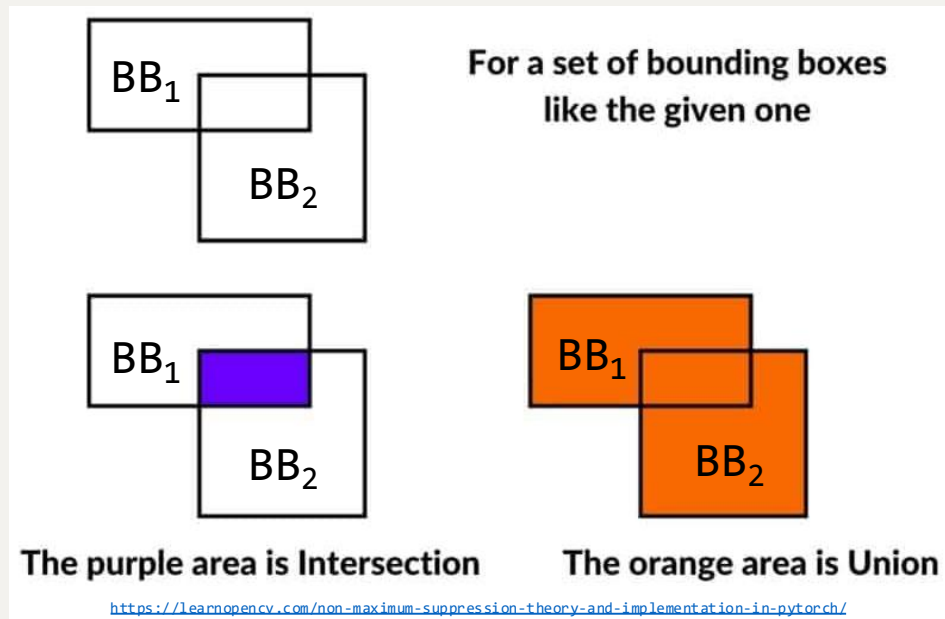
Bounding box

- Corner-based:
 - + upper-left (x_1, y_1) & lower-right corner (x_2, y_2)
- Center & size:
 - + center point (x, y) & width/height (w, h)
- Normalized center & size:
 - + center point $(x/W, y/H)$ & width/height $(w/W, h/H)$
- Anchor & offset:
 - + anchor as center & size (x_a, y_a, w_a, h_a)
 - + relative offsets (x_t, y_t, w_t, h_t) with
 $(x, y) = (x_a + x_t, y_a + y_t)$ & $(w, h) = (w_a + e^{w_t}, h_a + e^{h_t})$



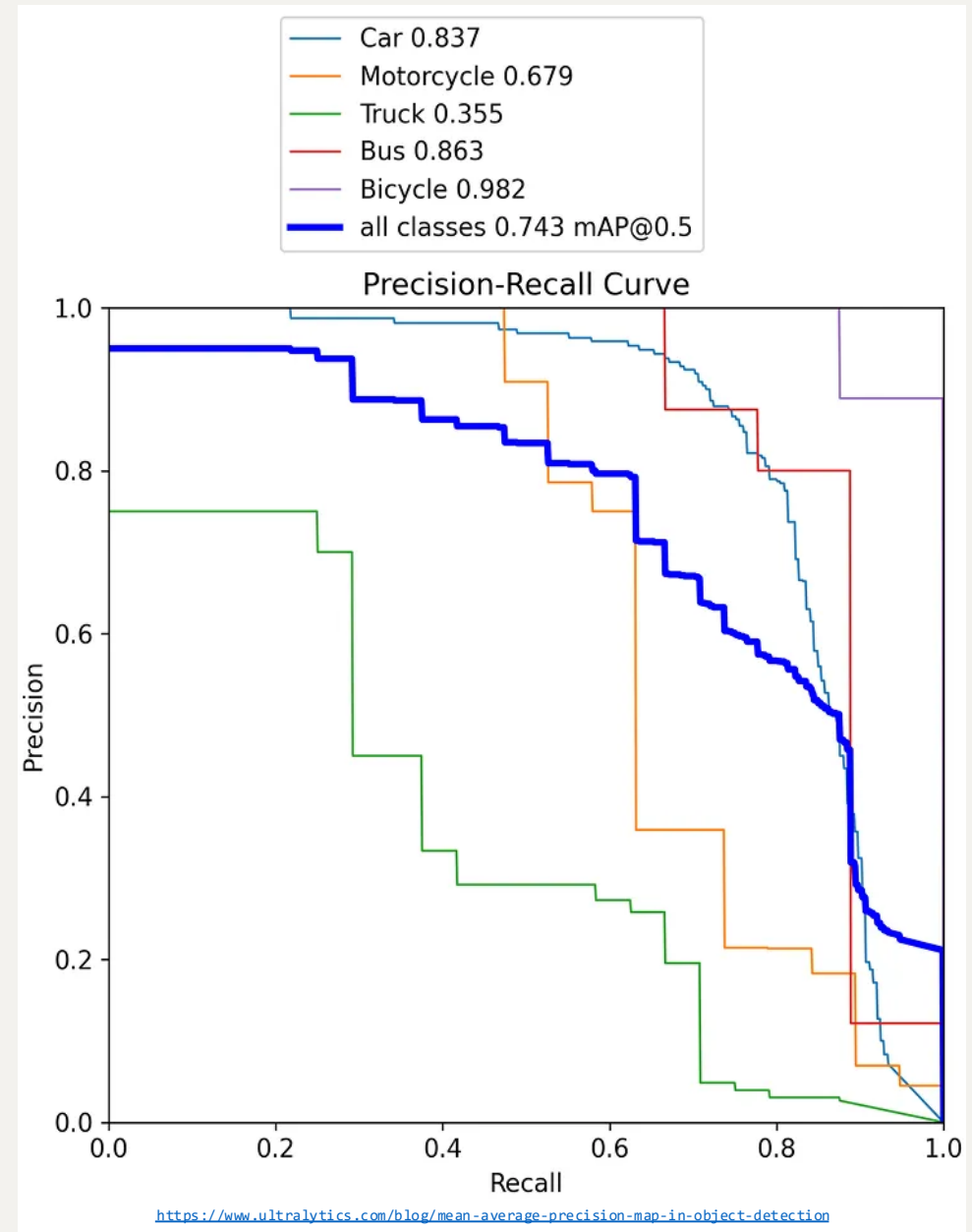
Intersection over Union

- $\text{IoU}(\text{BB}_1, \text{BB}_2) = A(\text{BB}_1 \cap \text{BB}_2) / A(\text{BB}_1 \cup \text{BB}_2)$
- An IoU of 0.5 or more is considered a good overlap.
- A picture is worth a thousand words:



mean Average Precision

- Precision-Recall Curve (PRC):
as recall increases, precision drops
- Average Precision (AP):
area under precision-recall curve (AUPRC)
- mean Average Precision (mAP):
scalar metric for model performance
- IoU threshold often indicated:
<mAP> @ <minIoU>



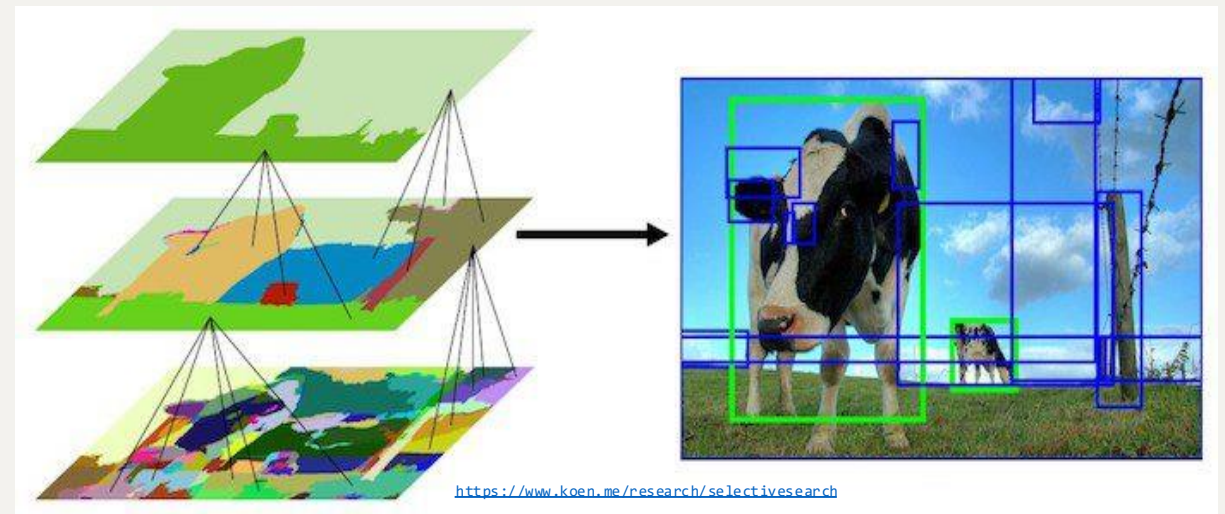
R-CNN: Region proposals

- Selective Search starts with oversegmentation

- Joining of similar segments:

$$s(r_1, r_2) = \alpha_1 * s_{\text{color}}(r_1, r_2) + \alpha_2 * s_{\text{texture}}(r_1, r_2) + \alpha_3 * s_{\text{size}}(r_1, r_2) + \alpha_4 * s_{\text{shape}}(r_1, r_2)$$

- + Color similarity:
color histogram intersection
- + Texture similarity:
texture histogram intersection
- + Size similarity
1-relative combined size
- + Shape similarity
1-bounding box overhead

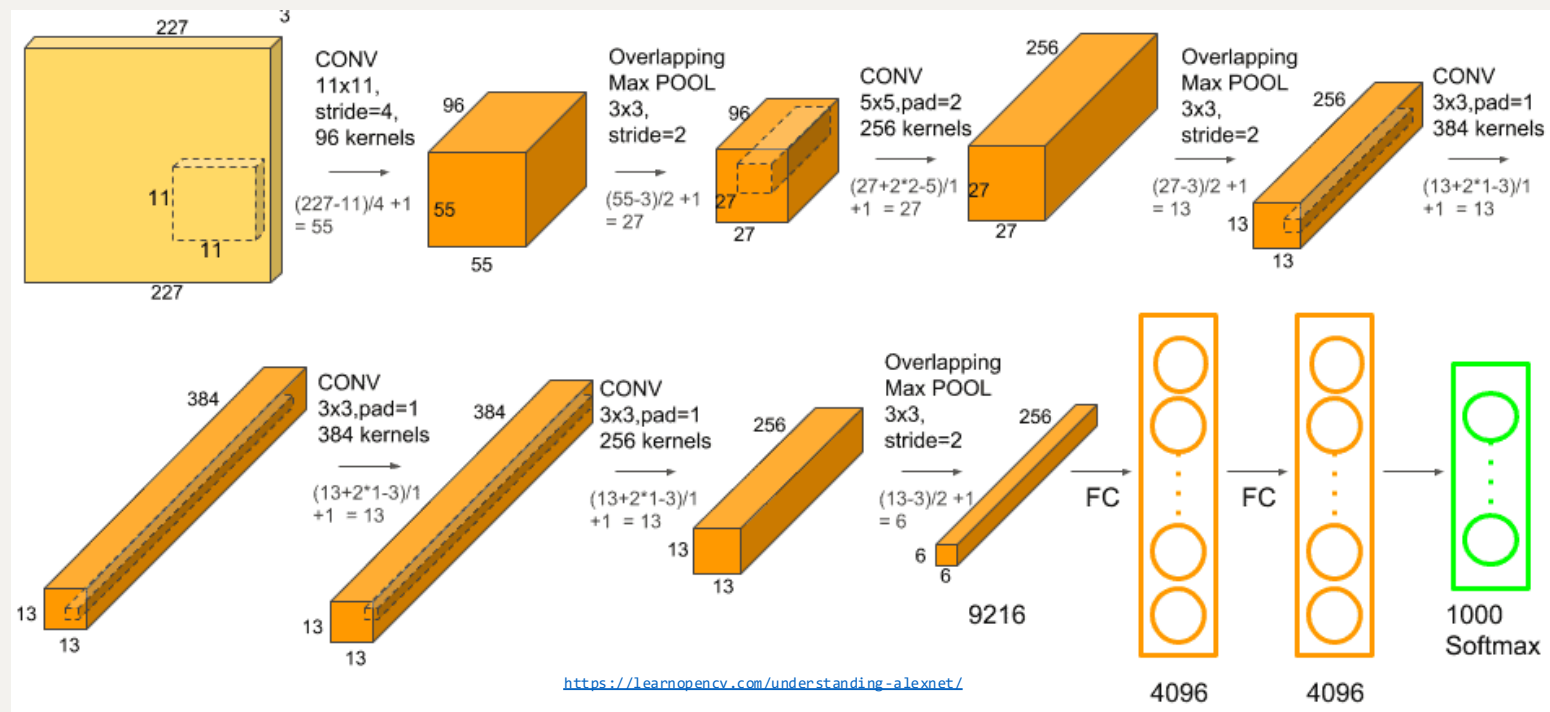


- Aiming for high recall → many spurious regions!



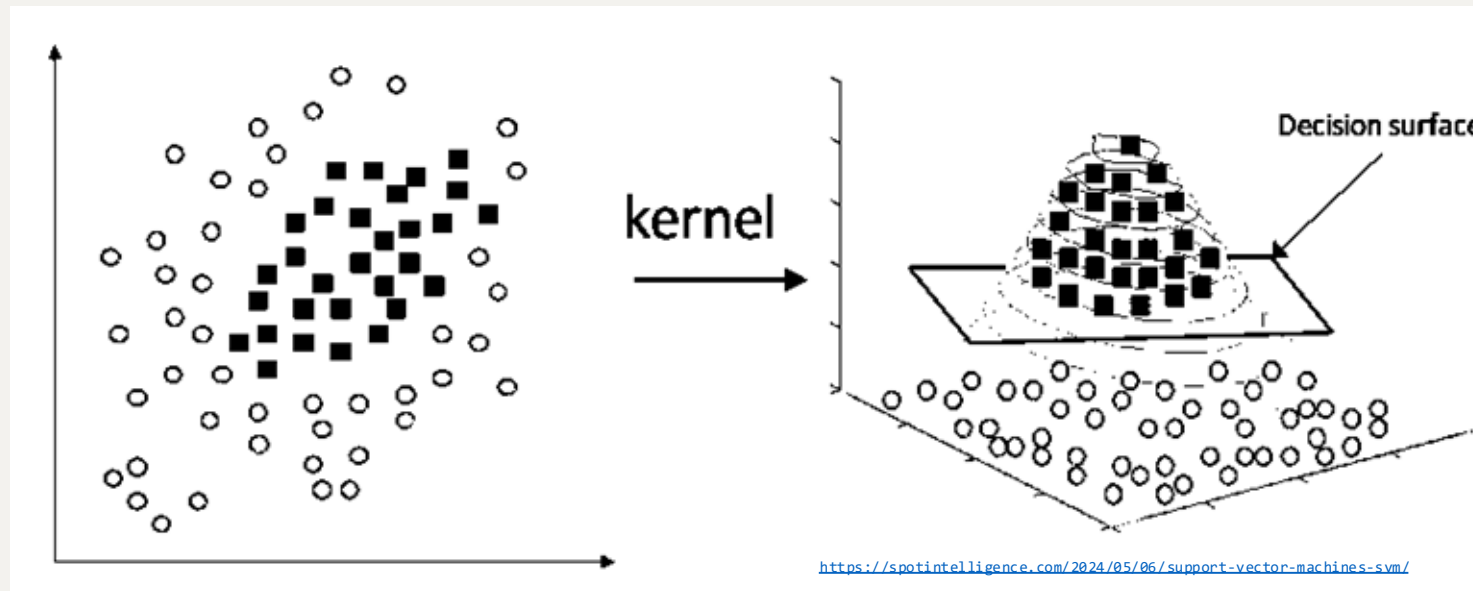
R-CNN: Feature extraction

- AlexNet: Convolutional Neural Network (8 convolutional, 2 fully-connected)
- Pretrained on ImageNet for classification challenge
- First vision model trained in a multi-GPU context ($\approx 60\text{M}$ parameters)



R-CNN: Region classification

- Super Vector Machines (SVMs)
- Supervised classification on features extracted through AlexNet
- Trained on a selection of positive and negative samples
- One SVM classifier per object class



R-CNN: Non-maximum suppression

- Multiple overlapping bounding boxes for the same object!
- Sort all bounding boxes by confidence
 1. Take bounding box with the highest confidence score.
 2. Remove strongly-overlapping ones according to an IoU threshold, typically $\text{IoU} \geq 0.3$.
 3. Repeat step 1.

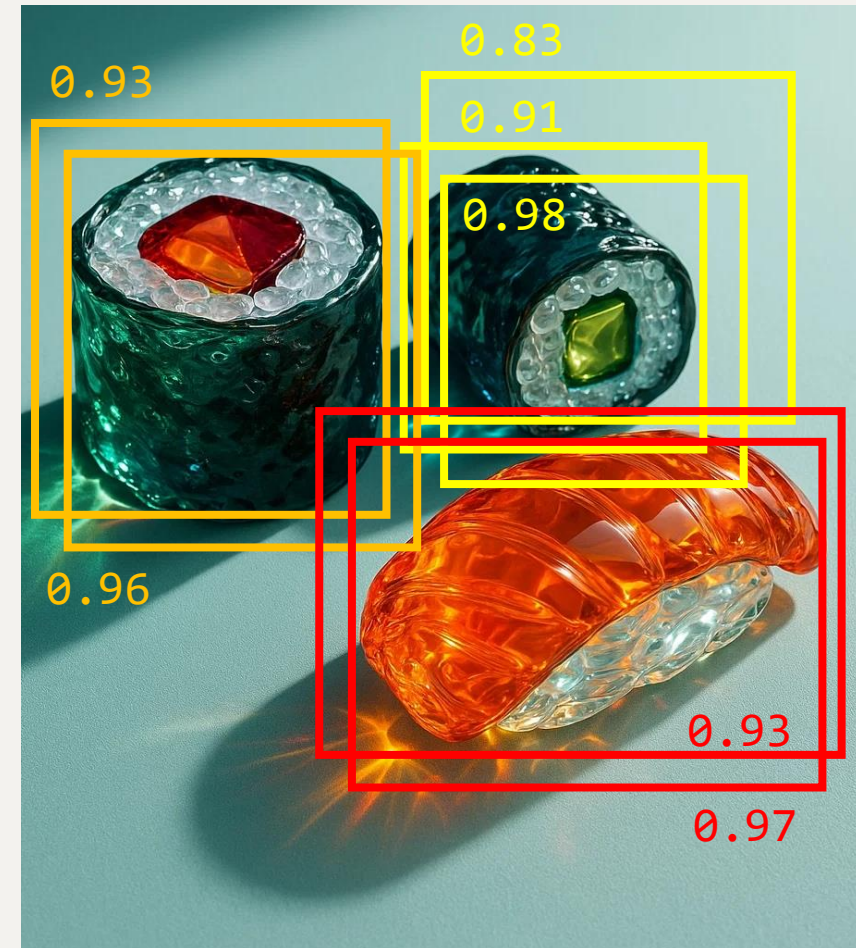


Applying
NMS



<https://learnopencv.com/non-maximum-suppression-theory-and-implementation-in-pytorch/>

LearnOpenCV.com



R-CNN: Bounding box offset regression

- Single linear layer predicts scale-invariant offsets (x_t, y_t, w_t, h_t) for center point and width/ height to adjust bounding boxes
- Trained on predictions $p = (x_p, y_p, w_p, h_p)$ and ground truths $g = (x_g, y_g, w_g, h_g)$.

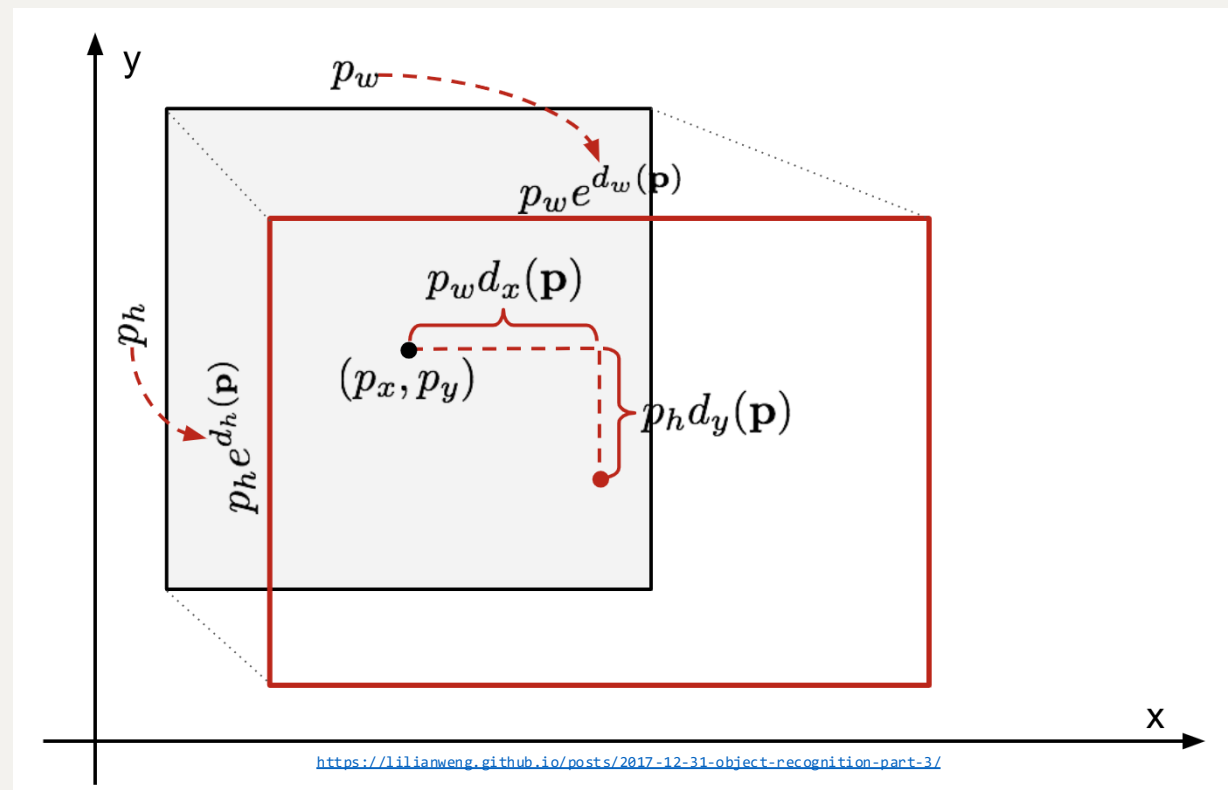
- $x_t = (x_g - x_p) / w_p$

- $y_t = (y_g - y_p) / h_p$

- $w_t = \log(w_g / w_p)$

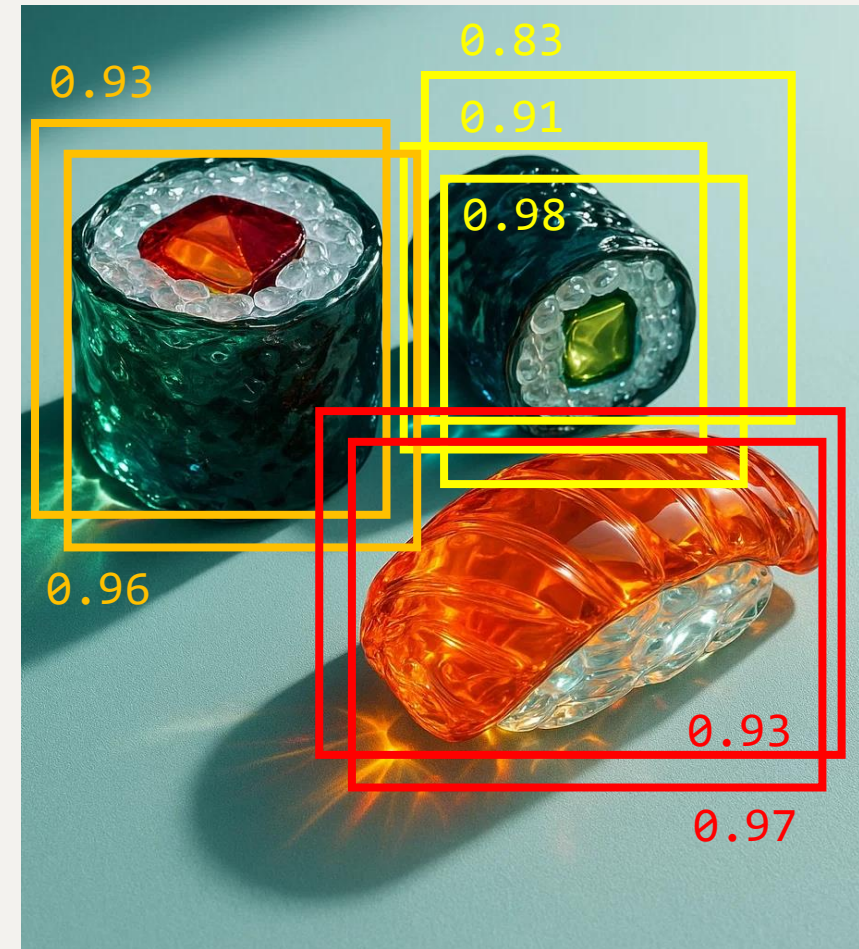
- $h_t = \log(h_g / h_p)$

- Loss function minimizes sum of squared errors of $g-p$



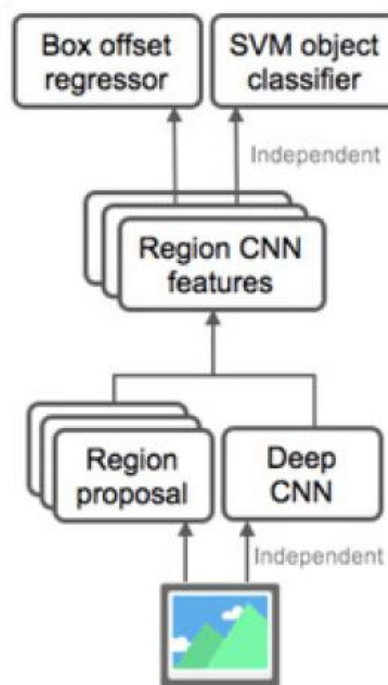
R-CNN: Putting it all together

1. Region proposals via Selective Search.
2. Feature extraction via AlexNet.
3. Object classification via SVMs.
4. Non-maximum suppression.
5. Bounding box offset regression.

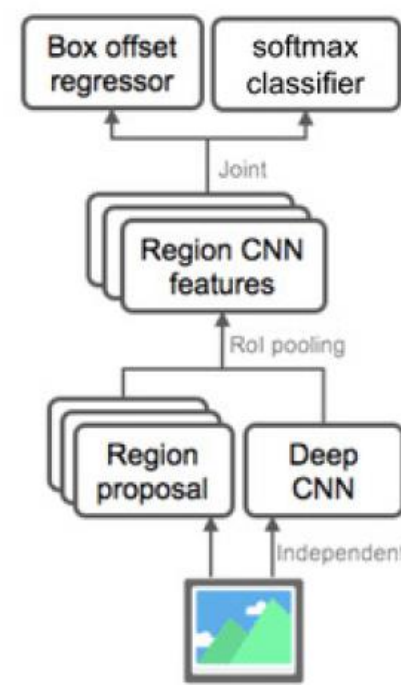


R-CNN: The fast & the furious

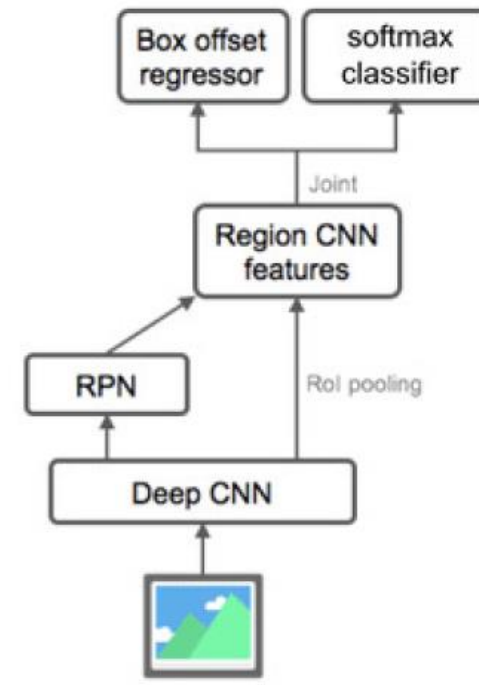
- R-CNN is a sloth!
 - + Numerous region proposals
 - + Feature extractor and classifier called for each region
 - + Separate box offset regressor & object classifier
- Fast R-CNN is a snail!
 - + Region-of-interest pooling for feature extractor & classifier
 - + Joint box offset regressor & object classifier
- Faster R-CNN is a turtle!
 - + Region proposal network
 - + RoI pooling all the way



R-CNN



Fast R-CNN



Faster R-CNN

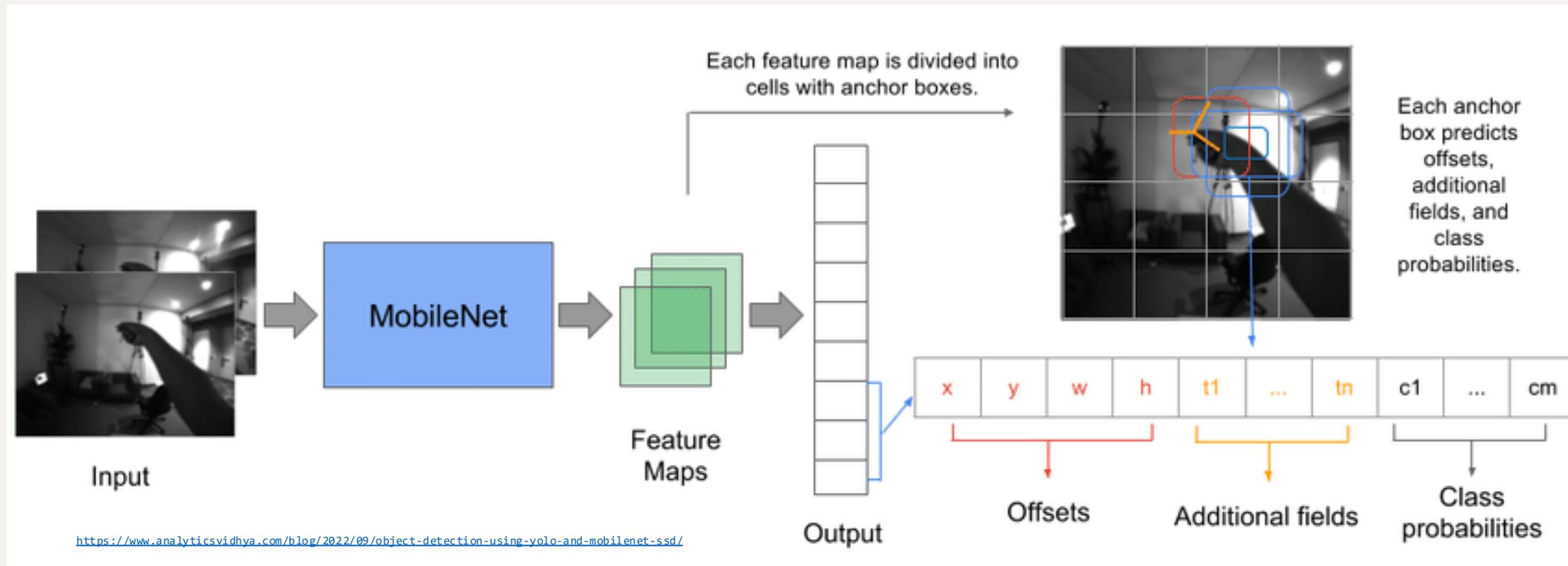


<https://www.ultralytics.com/blog/what-is-r-cnn-a-quick-overview>



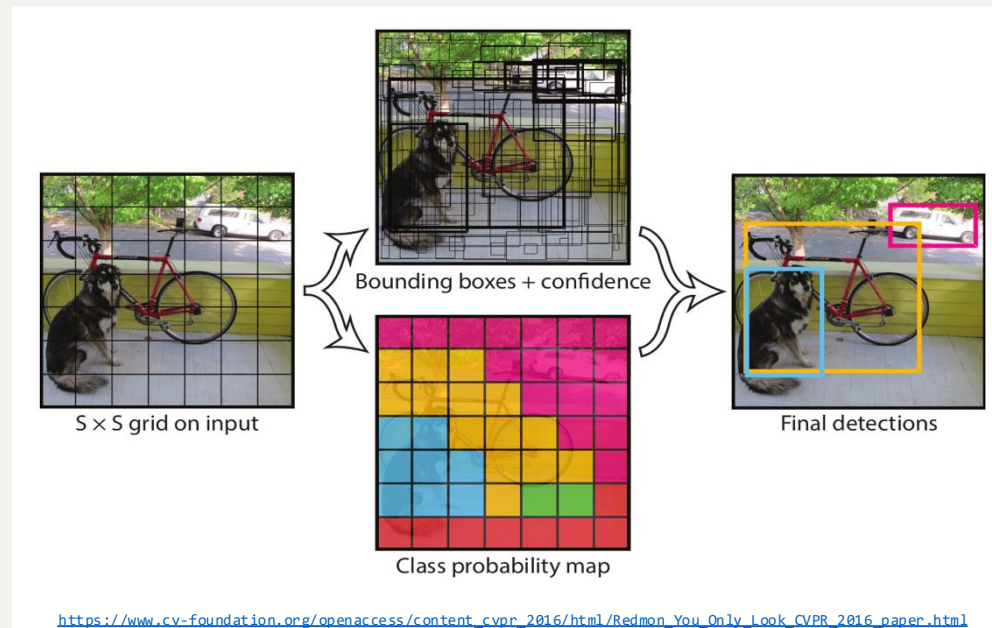
SSD: Single shot detection

- Motivated by R-CNN's inadequacy in constrained and real-time settings.
- Segment the image into $S \times S$ cells with anchor boxes.
- For each anchor box, predict offsets and class probabilities.



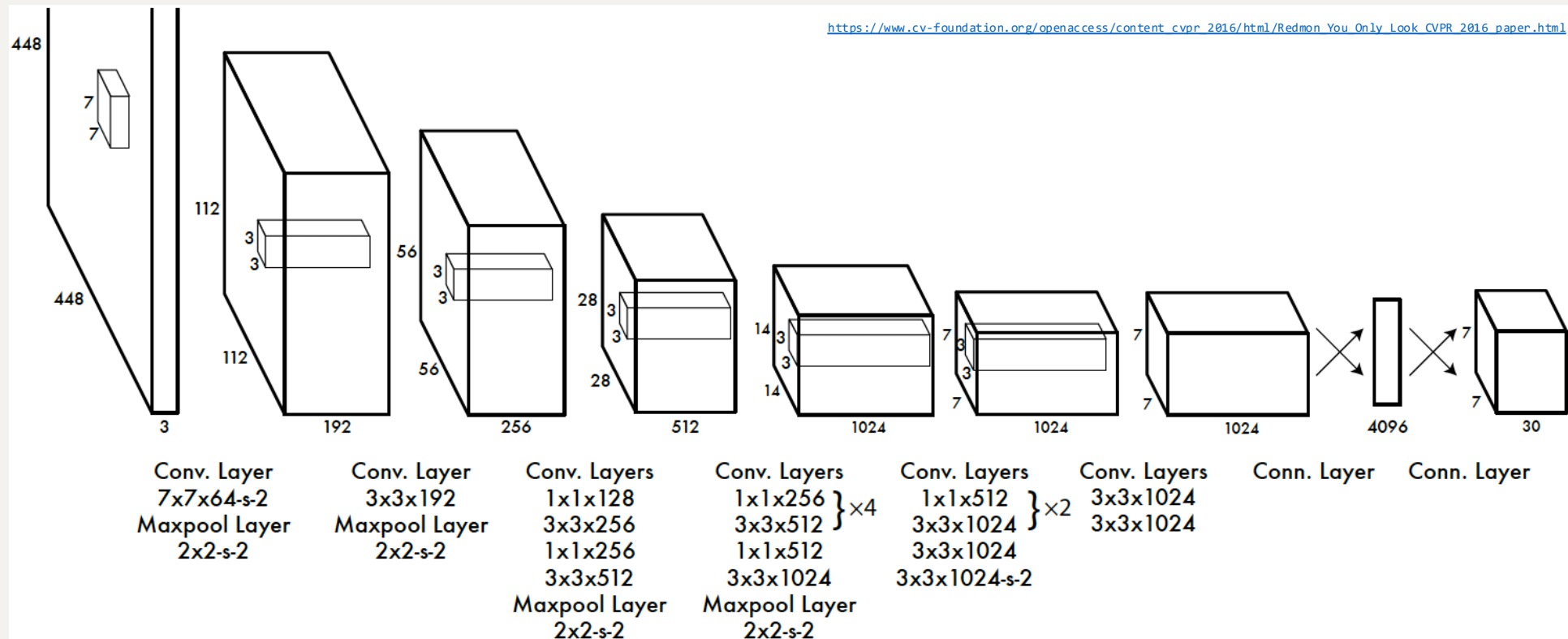
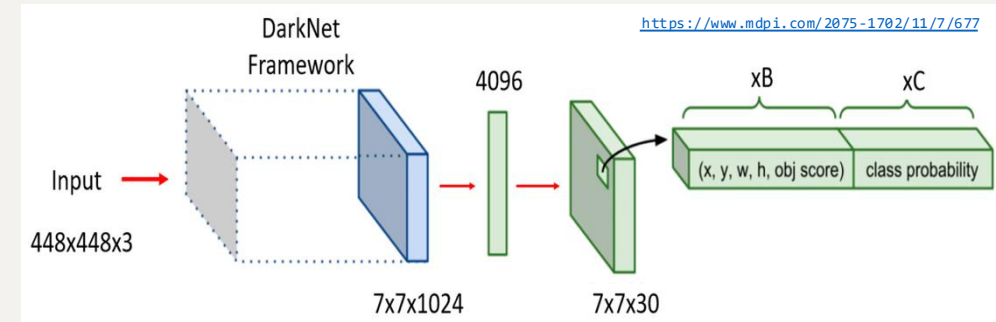
YOLO: You only look once

- Predicts multiple bounding boxes and class probabilities in one shot
- Reduces region proposals by orders of magnitude compared to R-CNN
- Segments image into $S \times S$ cells, predicts confidence per bounding box, & predicts class probabilities per cell
- Incorporates non-maximum suppression and bounding box offsets



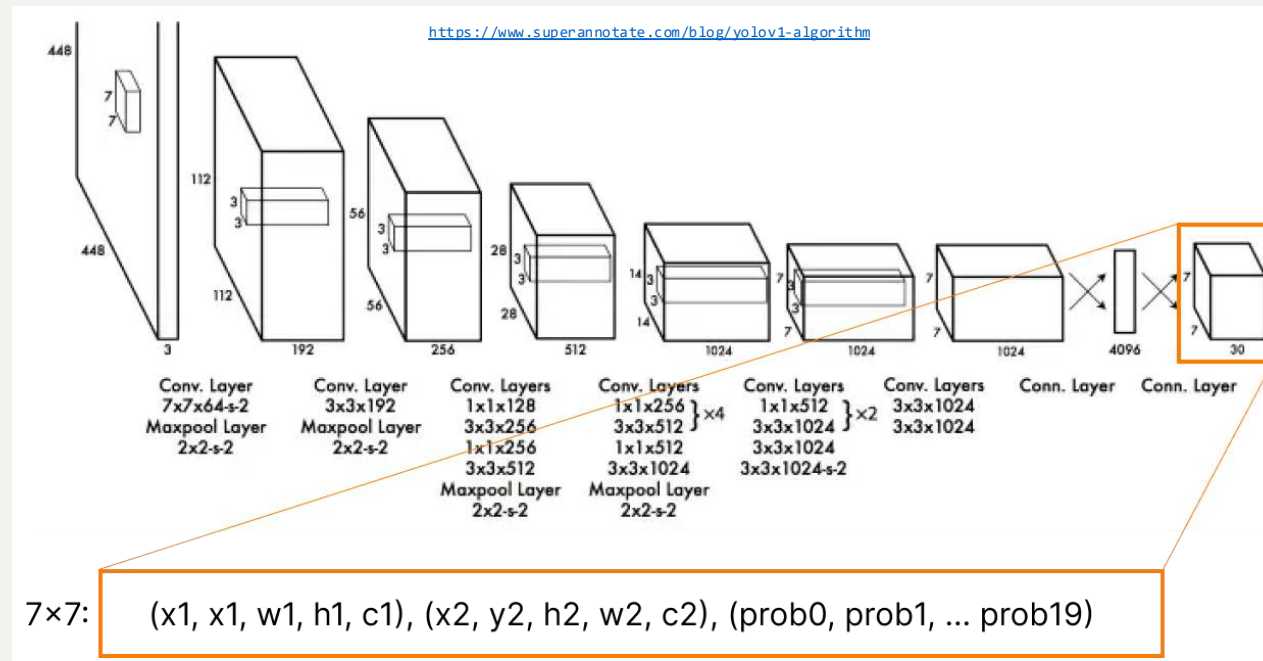
YOLO: Architecture

- CNN inspired by GoogLeNet
 - + 24 convolutional layers
 - + 2 fully-connected layers



YOLO: Prediction

- For each cell, predict $(p_1, \dots, p_C, \overbrace{c_1, x_1, y_1, w_1, h_1}^{BB_1}, \dots, \overbrace{c_B, x_B, y_B, w_B, h_B}^{BB_B})$
- Each cell can have up to B bounding boxes
- p_i is the probability of an object of class i being present in the current segment
- c_j is the predicted IoU of each BB with its corresponding ground truth



YOLO: Loss function

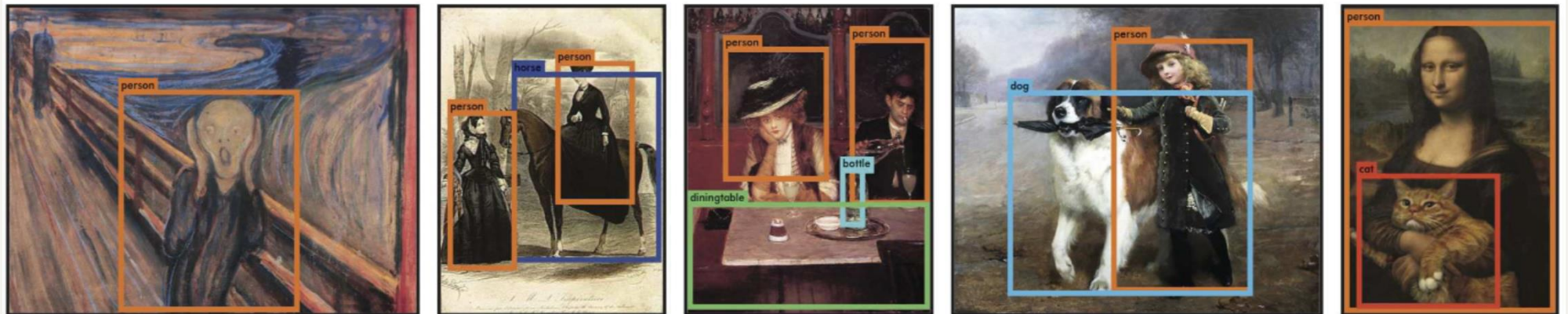
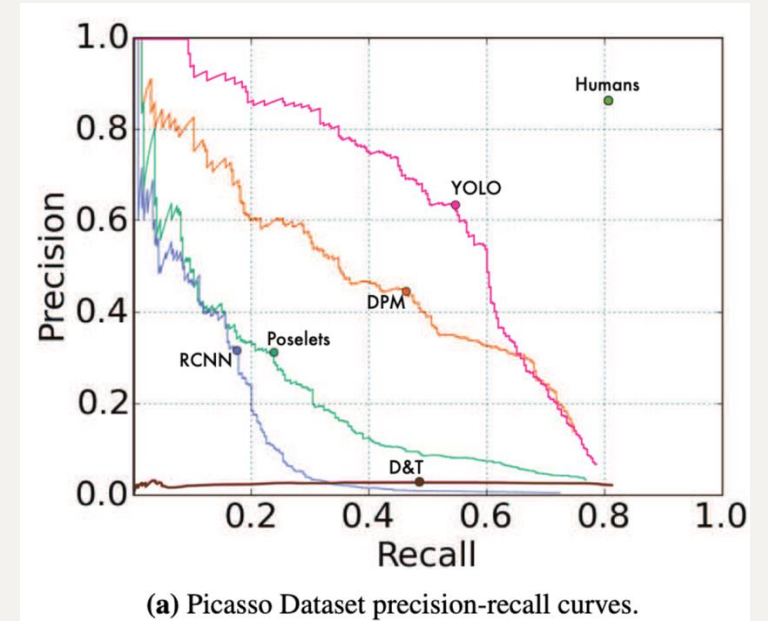
- Loss function to be optimized by SSE
- $1^{\text{obj}}_i = 1$ if any object center is in cell i
- $1^{\text{obj}}_{i,j} = 1$ if BB_j in cell i is the responsible predictor
- C_i is $\text{IoU}(p, g)$ for the responsible predictor, 0 otherwise
- Responsible predictor is BB_k with $k = \text{argmax}_1(\text{IoU}(\text{BB}_1, \text{BB}_g))$

$$\begin{aligned}
 & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
 & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
 & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
 & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
 & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
 \end{aligned}$$



YOLO: Putting it all together

1. Segmentation into $S \times S$ cells.
2. Joint prediction of confidence, class probabilities, & bounding boxes per cell.
3. Non-maximum suppresion.



YOLO: from v1 to v12

YOLOv1:
the original

YOLOv3:
multi-scale

YOLOv6: anchor-free
YOLOv7: aux. heads

YOLOv9: GLEAN backbone
YOLOv10: NMS-free
YOLOv11: refined backbone

2015

2016

2017

2018

2019

2020

2021

2022

2023

2024

2025

YOLOv2:
anchor boxes

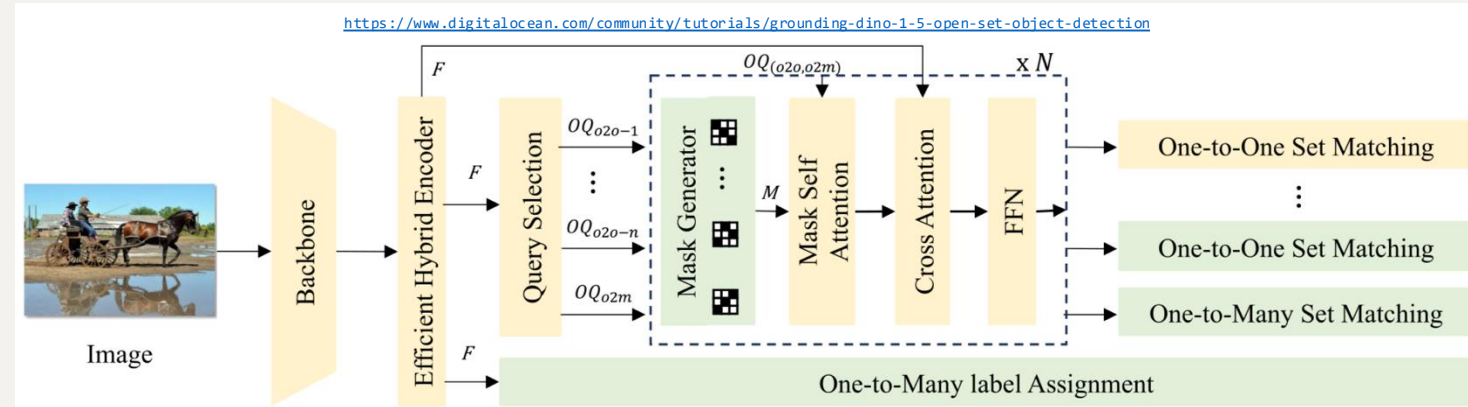
YOLOv4: Complete IoU
YOLOv5: auto anchor

YOLOv8:
decoupled
head

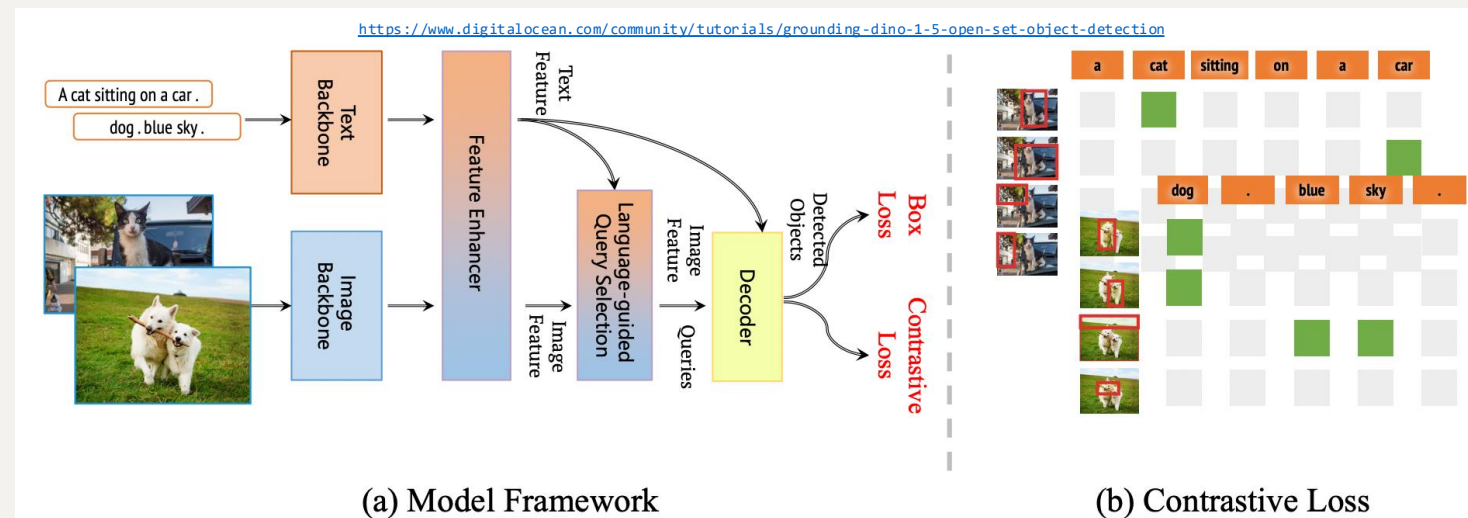
YOLOv12:
attention



Beyond YOLO



RT-DETRv3: fully transformers-based object detection



Grounding-DINO 1.5: open class object detection



Tasks of the day

Image classification

Object detection

Segmentation

Image generation

Inpainting

Image-text retrieval

Visual question answering

Video generation

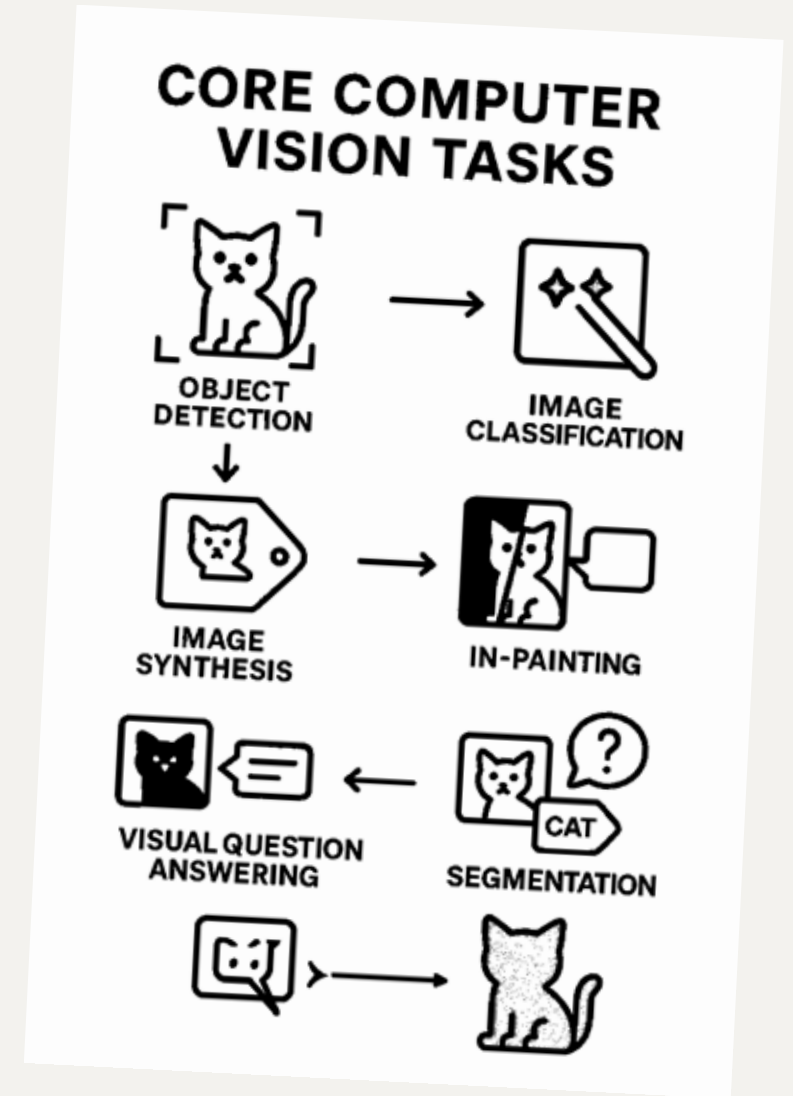


Image generation

GOAL:

Synthesize realistic images.

METRICS:

FID, KID, CLIPScore; human preference.

DATASETS:

LAION-5B, COCO, ImageNet.

ARCHITECTURES:

Diffusion, VAE, GAN.

